

Week 2 — Motion Detection & Tracking (Step-by-Step)

Week 2 — Motion Detection & Tracking (Step-by-Step Manual)

This guide takes you from a working camera to **per-fish activity metrics** written to **InfluxDB**, ready for

■ Objectives

- Detect moving fish via **OpenCV (MOG2)** background subtraction
- Track multiple targets with **temporary IDs**
- Compute **per-minute** movement metrics per fish
- Write metrics to **InfluxDB v2** for Grafana

0) Prerequisites (from Week 1)

- Raspberry Pi 4 with network access
- Camera Module 3 confirmed with `rpivid-test`
- Python 3 + OpenCV available (`python3-opencv` from apt is fine)
- InfluxDB + Grafana running

1) Create project folder & (optional) venv

Action: Organize code and isolate Python deps.

```
```bash
mkdir -p ~/fish-monitor
cd ~/fish-monitor
python3 -m venv .venv # optional
source .venv/bin/activate || true
pip install --upgrade pip
```
```

Context: Keeping things in one folder simplifies service setup later.

2) Install Python dependencies

Action: Add only what we need now.

```
```bash
OpenCV likely installed from apt; we add Influx client & utils
pip install influxdb-client==1.43.0 numpy==1.26.4 python-dotenv==1.0.1
(If you lack OpenCV Python, try: sudo apt install python3-opencv -y)
```
```

Context: We'll use `influxdb-client` to push metrics to InfluxDB v2.

3) Create a minimal camera test

Action: Verify OpenCV sees `/dev/video0` and can grab frames.

Create `cam_test.py`:

```
```python
import cv2

cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

ok, frame = cap.read()
print("Grab OK:", ok, "shape:", getattr(frame, 'shape', None))

cap.release()
```
```

Run: `python3 cam_test.py`

Context: If `ok` is False, run `v4l2-ctl --list-devices` and try the correct node (e.g., `/dev/video1`).

```

## 4) Motion detection + simple multi-target tracking
**Action:** Build a lean tracker using centroids and nearest-neighbor matching.

Create `motion_track.py`:
```python
import cv2, time, math, numpy as np
from collections import deque, defaultdict

--- Params ---
CAM_INDEX = 0
FRAME_W, FRAME_H = 1280, 720
MIN_CONTOUR_AREA = 200 # adjust to ignore bubbles/noise
MAX_ASSOC_DIST = 80 # px; max centroid distance to keep same ID
TRACE_LEN = 12 # for on-screen trails

Background subtractor
bg = cv2.createBackgroundSubtractorMOG2(history=300, varThreshold=16, detectShadows=True)

Track state
next_id = 1
tracks = {} # id -> {'centroid': (x,y), 'trace': deque}
last_seen = {} # id -> timestamp
per_minute_dist = defaultdict(float) # id -> pixels moved this minute
last_pos_for_dist = {} # id -> (x,y)
minute_started = time.time()

cap = cv2.VideoCapture(CAM_INDEX)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, FRAME_W)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_H)

def centroid_of(cnt):
 x, y, w, h = cv2.boundingRect(cnt)
 return (int(x + w/2), int(y + h/2)), (x, y, w, h)

def euclid(a, b):
 return math.hypot(a[0]-b[0], a[1]-b[1])

while True:
 ok, frame = cap.read()
 if not ok:
 break

 fg = bg.apply(frame)
 # Clean up mask
 fg = cv2.medianBlur(fg, 5)
 _, th = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY)
 th = cv2.dilate(th, np.ones((3,3), np.uint8), iterations=2)

 cnts, _ = cv2.findContours(th, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
 detections = []
 for c in cnts:
 if cv2.contourArea(c) < MIN_CONTOUR_AREA:
 continue
 ctr, box = centroid_of(c)
 detections.append((ctr, box))

 # Associate detections to existing tracks (greedy nearest-neighbor)
 unmatched_tracks = set(tracks.keys())
 assignments = [] # (track_id, ctr, box)

 for ctr, box in detections:
 # find nearest track
 best_id, best_d = None, 1e9
 for tid in list(unmatched_tracks):
 d = euclid(ctr, tracks[tid]['centroid'])
 if d < best_d:
 best_d, best_id = d, tid
 if best_id is not None and best_d <= MAX_ASSOC_DIST:
 assignments.append((best_id, ctr, box))
 unmatched_tracks.discard(best_id)
 else:
 assignments.append((None, ctr, box))

 # Update existing & create new tracks
 next_id_local = False
 for tid, ctr, box in assignments:

```

```

 if tid is None:
 tid = next_id
 next_id += 1
 tracks[tid] = {'centroid': ctr, 'trace': deque(maxlen=TRACE_LEN)}
 last_pos_for_dist[tid] = ctr
 # distance accumulation
 prev = last_pos_for_dist.get(tid, ctr)
 per_minute_dist[tid] += euclid(prev, ctr)
 last_pos_for_dist[tid] = ctr

 tracks[tid]['centroid'] = ctr
 tracks[tid]['trace'].append(ctr)
 last_seen[tid] = time.time()

 # draw
 x,y,w,h = box
 cv2.rectangle(frame, (x,y), (x+w,y+h), (0,255,0), 2)
 cv2.putText(frame, f"ID {tid}", (x, y-6), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 1)

 # prune stale tracks (not seen for 3s)
 now = time.time()
 for tid in list(tracks.keys()):
 if now - last_seen.get(tid, now) > 3.0:
 tracks.pop(tid, None)
 last_pos_for_dist.pop(tid, None)

 # draw traces
 for tid, st in tracks.items():
 for i in range(1, len(st['trace'])):
 cv2.line(frame, st['trace'][i-1], st['trace'][i], (255, 255, 0), 2)

 cv2.imshow("Motion Tracking", frame)
 if cv2.waitKey(1) & 0xFF == ord('q'):
 break

```

```

cap.release()
cv2.destroyAllWindows()
```

```

****Context:**** This isn't perfect identity tracking, but it's enough to generate ****per-minute movement**** signals

5) Write per-minute metrics to InfluxDB
****Action:**** Add a writer that flushes distance totals each minute.

1) In InfluxDB (UI at `http://localhost:8086`), create:

- ****Org:**** e.g., `home`
- ****Bucket:**** e.g., `fish`
- ****Token:**** create a read/write API token

2) Create `.env` in `~/fish-monitor`:

```

```bash
INFLUX_URL=http://localhost:8086
INFLUX_TOKEN=REPLACE_ME
INFLUX_ORG=home
INFLUX_BUCKET=fish
```

```

3) Create `motion_track_influx.py`:

```

```python
import os, time, math, cv2, numpy as np
from collections import deque, defaultdict
from influxdb_client import InfluxDBClient, Point, WritePrecision
from dotenv import load_dotenv

```

```

--- Load env ---
try:
 load_dotenv()
except Exception:
 pass

```

```

URL = os.getenv("INFLUX_URL", "http://localhost:8086")
TOKEN = os.getenv("INFLUX_TOKEN", "YOUR_TOKEN")
ORG = os.getenv("INFLUX_ORG", "home")
BUCKET = os.getenv("INFLUX_BUCKET", "fish")

```

```

client = InfluxDBClient(url=URL, token=TOKEN, org=ORG)
writer = client.write_api(write_options=None)

--- CV params ---
CAM_INDEX = 0
FRAME_W, FRAME_H = 1280, 720
MIN_CONTOUR_AREA = 200
MAX_ASSOC_DIST = 80
TRACE_LEN = 12

bg = cv2.createBackgroundSubtractorMOG2(history=300, varThreshold=16, detectShadows=True)

next_id = 1
tracks = {}
last_seen = {}
per_minute_dist = defaultdict(float)
last_pos_for_dist = {}
minute_started = time.time()

def euclid(a,b):
 return math.hypot(a[0]-b[0], a[1]-b[1])

def centroid_of(cnt):
 x,y,w,h = cv2.boundingRect(cnt)
 return (int(x+w/2), int(y+h/2)), (x,y,w,h)

cap = cv2.VideoCapture(CAM_INDEX)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, FRAME_W)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, FRAME_H)

def flush_minute(ts):
 global per_minute_dist
 for tid, dist_px in per_minute_dist.items():
 p = Point("fish_activity").tag("fish_id", str(tid)).field("distance_px", float(dist_px)).time(int(ts))
 writer.write(bucket=BUCKET, org=ORG, record=p)
 per_minute_dist = defaultdict(float)

while True:
 ok, frame = cap.read()
 if not ok: break

 fg = bg.apply(frame)
 fg = cv2.medianBlur(fg, 5)
 _, th = cv2.threshold(fg, 200, 255, cv2.THRESH_BINARY)
 th = cv2.dilate(th, np.ones((3,3), np.uint8), 2)

 cnts, _ = cv2.findContours(th, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
 dets = []
 for c in cnts:
 if cv2.contourArea(c) < MIN_CONTOUR_AREA: continue
 ctr, box = centroid_of(c)
 dets.append((ctr, box))

 unmatched = set(tracks.keys())
 assigns = []
 for ctr, box in dets:
 best_id, best_d = None, 1e9
 for tid in list(unmatched):
 d = euclid(ctr, tracks[tid]['centroid'])
 if d < best_d:
 best_d, best_id = d, tid
 if best_id is not None and best_d <= MAX_ASSOC_DIST:
 assigns.append((best_id, ctr, box))
 unmatched.discard(best_id)
 else:
 assigns.append((None, ctr, box))

 import time
 for tid, ctr, box in assigns:
 if tid is None:
 tid = next_id; next_id += 1
 tracks[tid] = {'centroid': ctr, 'trace': deque(maxlen=TRACE_LEN)}
 last_pos_for_dist[tid] = ctr
 prev = last_pos_for_dist.get(tid, ctr)
 per_minute_dist[tid] += euclid(prev, ctr)

```

```

 last_pos_for_dist[tid] = ctr
 tracks[tid]['centroid'] = ctr
 tracks[tid]['trace'].append(ctr)
 last_seen[tid] = time.time()

 now = time.time()
 for tid in list(tracks.keys()):
 if now - last_seen.get(tid, now) > 3.0:
 tracks.pop(tid, None)
 last_pos_for_dist.pop(tid, None)

 if now - minute_started >= 60.0:
 flush_minute(now)
 minute_started = now

 if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release()
flush_minute(time.time())
client.close()
print("Stopped and flushed.")
```

**Context:** We send one row per fish ID each minute (`measurement=fish_activity, tag=fish_id, field=distance_px`)

---

## 6) Grafana – add data source & panels
**Action:** In Grafana (http://pi:3000):
1. **Add Data Source → InfluxDB** (v2):
    - URL: `http://localhost:8086`
    - Organization: `home`
    - Token: your API token
    - Default Bucket: `fish`
2. **Create a Panel** with a Flux query like:
    ```flux
 from(bucket: "fish")
 > range(start: -6h)
 > filter(fn: (r) => r._measurement == "fish_activity")
 > filter(fn: (r) => r._field == "distance_px")
 > aggregateWindow(every: 1m, fn: mean, createEmpty: false)
 > yield(name: "mean")
    ```
3. Duplicate the panel and **filter by `fish_id`** to create one per fish.

**Alerts:** Add a threshold rule like “distance_px mean < X for 10m”.

---

## 7) Run on boot (systemd service)
**Action:** Create a service to auto-start tracking.

Create `/etc/systemd/system/fish-track.service`:
```ini
[Unit]
Description=Fish Motion Tracking
After=network.target

[Service]
User=pi
WorkingDirectory=/home/pi/fish-monitor
EnvironmentFile=/home/pi/fish-monitor/.env
ExecStart=/usr/bin/python3 /home/pi/fish-monitor/motion_track_influx.py
Restart=always

[Install]
WantedBy=multi-user.target
```

Enable & start:
```bash
sudo systemctl daemon-reload
sudo systemctl enable fish-track
sudo systemctl start fish-track
sudo systemctl status fish-track
```

```

8) Tuning & Tips

- Start with **MIN_CONTOUR_AREA = 200–500**; increase if bubbles cause noise.
- If tracks “jump”, **lower MAX_ASSOC_DIST** or reduce frame rate.
- Consider downscaling to **1280×720 @ 15–20 fps** for stability and low CPU.
- Keep lighting consistent to avoid flicker-triggered motion.

9) What “good” looks like

- InfluxDB gets **60 points per hour per active track**.
- Grafana charts show distinct activity lines; an inactive fish is visually obvious.
- Alerts trigger only when a fish **sustained-inactive** (>10 min) vs baseline.